

Smart City Incident Management System

Combined Project Document

Requirements Specification (R#1) + Relational Schema & Data Dictionary (R#3)

PART 1 — Business & System Requirements Specification (R#1)

1. Project Overview

The **Smart City Incident Management System** is a centralized database system designed to help a city receive, organize, assign, monitor, and resolve reports of incidents.

Citizens can use the system to report problems or emergencies such as:

- Traffic accidents
- Water leaks
- Power outages
- Damaged roads
- Broken streetlights
- Garbage collection problems
- Fire emergencies
- Flooding
- Public safety complaints
- Other city-service incidents

After a citizen submits an incident, the system allows authorized employees to **validate it, classify its severity and priority, assign it to an appropriate response team, monitor its progress, and record its resolution.**

The system also keeps historical information about incidents so that city administrators and department managers can analyze performance and generate reports.

Main objectives

The system is designed to:

1. Make incident reporting easier for citizens.
2. Keep all incident information in one centralized database.
3. Allow operators to validate submitted incidents.
4. Help determine the appropriate department and response team.

5. Track the status of every incident from reporting until closure.
 6. Notify citizens about important changes.
 7. Allow response teams to record their work and upload evidence.
 8. Track employees, teams, skills, vehicles, and equipment.
 9. Collect citizen feedback after incidents are resolved.
 10. Provide performance information and reports for management.
 11. Maintain historical records for future analysis.
 12. Control access to the system using user accounts and roles.
-

2. Stakeholders and Their Requirements

A **stakeholder** is a person or group that interacts with the system or benefits from its services.

The main stakeholders are:

1. Citizen
 2. Operator
 3. Response Team / Team Members
 4. Department Manager
 5. City Administrator
-

2.1 Citizen

Citizens are the people who report incidents to the city.

BR1 — Report an Incident

Requirement:

Citizens shall be able to report a city incident by providing information such as the incident location and incident type.

Type: Functional

Related entities:

CITIZEN → INCIDENT → INCIDENT_LOCATION

INCIDENT → INCIDENT_TYPE

Explanation

A citizen creates an incident report. Each incident records who reported it, what type of incident it is, and where it happened.

BR2 — Attach Evidence

Requirement:

Citizens shall be able to attach one or more photos as supporting evidence when reporting an incident.

Type: Functional

Related entities:

INCIDENT → INCIDENT_EVIDENCE

Explanation

An incident can have multiple pieces of evidence. For example, a citizen reporting a damaged road could upload several photos showing the damage.

INCIDENT_EVIDENCE is modeled as a **weak entity** because the evidence belongs to a specific incident.

BR3 — Track Incident Status

Requirement:

Citizens shall be able to track the current status of any incident they have reported.

Type: Functional

Related entities:

INCIDENT → INCIDENT_STATUS

INCIDENT → INCIDENT_STATUS_HISTORY

Explanation

Every incident has a current status such as:

- Reported
- Validated
- Assigned
- In Progress

- Resolved
- Closed

The system also keeps a history of previous statuses so that the complete progress of the incident can be reviewed later.

BR4 — Receive Status Notifications

Requirement:

Citizens shall automatically receive a notification whenever the status of their reported incident changes.

Type: Functional

Related entities:

INCIDENT → NOTIFICATION

CITIZEN → NOTIFICATION

Explanation

When an important change occurs, such as an incident being assigned or resolved, the system can create a notification for the citizen who reported it.

BR5 — Provide Feedback

Requirement:

Citizens shall be able to rate the resolution of an incident and leave a comment once the incident is closed.

Type: Functional

Related entities:

CITIZEN → CITIZEN_FEEDBACK

INCIDENT → CITIZEN_FEEDBACK

Explanation

After an incident has been resolved, the citizen can provide a rating from **1 to 5** and optionally write a comment.

This feedback can later be used to evaluate service quality.

2.2 Operator

Operators are employees responsible for reviewing and processing newly submitted incidents.

BR6 — Validate Incidents

Requirement:

Operators shall be able to review newly submitted incidents and mark them as validated or rejected.

Type: Functional

Related entities:

OPERATOR → VALIDATION

VALIDATION → INCIDENT

Explanation

An operator checks whether a submitted incident is valid.

The system records:

- Which incident was reviewed
 - Which operator reviewed it
 - When it was reviewed
 - Whether it was accepted or rejected
 - Any notes provided by the operator
-

BR7 — Classify Severity and Priority

Requirement:

Operators shall be able to classify a validated incident according to its severity and priority.

Type: Functional

Related entities:

INCIDENT → SEVERITY

INCIDENT → PRIORITY

Explanation

Severity describes how serious the incident is.

Example:

- Low
- Medium
- High
- Critical

Priority determines how urgently the incident should be handled.

This allows the response process to focus on the most important incidents first.

BR8 — Assign a Response Team

Requirement:

Operators shall be able to assign a validated incident to the appropriate response team.

Type: Functional

Related entities:

OPERATOR → INCIDENT_ASSIGNMENT

INCIDENT → INCIDENT_ASSIGNMENT

RESPONSE_TEAM → INCIDENT_ASSIGNMENT

Explanation

The `INCIDENT_ASSIGNMENT` table records the assignment process.

It stores information such as:

- Incident
- Response team
- Department
- Operator who made the assignment
- Assignment date/time
- Acceptance time
- Completion time
- Assignment status
- Assignment method
- Assignment notes

BR9 — Suggest an Appropriate Department

Requirement:

The system shall automatically suggest a department based on the incident category to speed up the assignment process.

Type: Functional

Related entities:

INCIDENT_CATEGORY ↔ DEPARTMENT

resolved through:

CATEGORY_DEPARTMENT

Explanation

Different incident categories can be handled by different departments.

For example:

- Road damage → Roads Department
- Water leak → Water Department
- Power outage → Electricity Department
- Garbage problem → Waste Management Department

The CATEGORY_DEPARTMENT table allows one category to be associated with one or more departments.

2.3 Response Team and Team Members

Response teams are responsible for physically handling incidents after they are assigned.

BR10 — Assignment Notification

Requirement:

Response teams shall be notified immediately when an incident is assigned to them.

Type: Functional

Related entity:

RESPONSE_TEAM → INCIDENT_ASSIGNMENT

Explanation

When an operator assigns an incident to a team, the team needs to know that a new task has been assigned.

BR11 — Update Incident Progress

Requirement:

Response teams shall be able to update the progress and status of an assigned incident.

Type: Functional

Related entity:

INCIDENT_ASSIGNMENT

Important attributes include:

- AssignmentStatus
- AcceptedDateTime
- CompletionDateTime

Explanation

The team can show whether the assignment is still pending, accepted, in progress, or completed.

The dates allow the system to measure how long the response process took.

BR12 — Upload Completion Evidence

Requirement:

Response teams shall be able to upload evidence such as photos or documents proving that work has been completed.

Type: Functional

Related entity:

INCIDENT_EVIDENCE

Explanation

For example, if a team repairs a damaged streetlight, it can upload a photograph showing the completed repair.

The same evidence structure can therefore support evidence submitted by citizens as well as evidence related to the response process.

BR13 — Maintain Team Members

Requirement:

Each response team shall maintain a roster of its members.

Type: Functional

Related entities:

RESPONSE_TEAM ↔ EMPLOYEE

resolved through:

TEAM_MEMBER

Explanation

An employee can belong to a response team, and a response team can contain multiple employees.

Because this represents an **M:N relationship**, the `TEAM_MEMBER` table is used to connect employees and teams.

It can also record:

- Join date
- Leave date
- Team role

BR13a — Record Team Skills

Requirement:

Response teams shall be able to record their capabilities and skills.

Type: Functional

Related entities:

RESPONSE_TEAM ↔ TEAM_SKILL

resolved through:

TEAM_HAS_SKILL

Explanation

A team may have several skills, and the same skill may belong to several teams.

For example, a team could have:

- Electrical repair
- Road maintenance
- Flood response
- Fire response

TEAM_HAS_SKILL resolves this M:N relationship.

2.4 Department Manager

A Department Manager is modeled as an **employee with an appropriate role**, rather than as a separate entity.

The system uses:

USER_ACCOUNT + ROLE + ROLE_ASSIGNMENT

to manage staff roles.

The manager is associated with a department and can monitor the teams and incidents belonging to that department.

BR14 — Monitor Department Workload

Requirement:

Department managers shall be able to monitor the current workload of their department, including open, assigned, and in-progress incidents.

Type: Functional

Related entities:

DEPARTMENT → RESPONSE_TEAM → INCIDENT_ASSIGNMENT

Explanation

The manager can use assignment information to determine how many incidents are currently being handled by the department and its teams.

BR15 — Monitor Response and Resolution Times

Requirement:

Department managers shall be able to review average response and resolution times for their department's incidents.

Type: Reporting

Related entity:

PERFORMANCE_RECORD

The calculations are based on incident dates such as:

- ReportedDateTime
- ResolvedDateTime

Explanation

The system can calculate performance measurements from the stored incident and assignment dates.

For example:

Resolution Time = ResolvedDateTime – ReportedDateTime

These measurements help managers understand how efficiently incidents are being handled.

BR16 — Generate Performance Reports

Requirement:

Department managers shall be able to generate performance reports for their department.

Type: Reporting

Related entities:

DEPARTMENT → PERFORMANCE_REPORT

EMPLOYEE → PERFORMANCE_REPORT

Explanation

A performance report can summarize information for a selected department and time period.

2.5 City Administrator

City Administrators are responsible for managing the overall system and its organizational data.

BR17 — Manage Departments

Requirement:

City administrators shall be able to create, update, and deactivate departments.

Type: Functional

Related entity:

CITY_ADMINISTRATOR → DEPARTMENT

Explanation

The administrator maintains the city's organizational structure.

BR18 — Manage User Accounts

Requirement:

City administrators shall be able to manage user accounts for citizens, operators, response team members, and staff.

Type: Functional

Related entities:

USER_ACCOUNT

ROLE

ROLE_ASSIGNMENT

Explanation

Instead of creating a separate login system for every user type, the system uses a shared account and role structure.

A user account can be assigned one or more roles.

BR19 — Manage Reference Data

Requirement:

City administrators shall be able to configure system reference data including incident categories, incident types, severity levels, priority levels, and status values.

Type: Functional

Related entities:

- INCIDENT_CATEGORY
- INCIDENT_TYPE
- SEVERITY
- PRIORITY
- INCIDENT_STATUS

Explanation

These tables contain standard values used throughout the system.

This avoids repeatedly storing the same text in incident records.

BR20 — View System-Wide Dashboards

Requirement:

City administrators shall be able to view dashboards summarizing incidents, response teams, and departments across the entire city.

Type: Reporting

Related entities:

- INCIDENT
- DEPARTMENT
- RESPONSE_TEAM
- PERFORMANCE_RECORD

Explanation

Dashboards can provide information such as:

- Number of incidents

- Incidents by category
 - Incidents by department
 - Open incidents
 - Resolved incidents
 - Team performance
 - Average response time
 - Average resolution time
-

BR21 — Manage Employees

Requirement:

City administrators shall be able to manage employees and their department assignments.

Type: Functional

Related entity:

EMPLOYEE

Explanation

The administrator can maintain employee information and associate employees with the appropriate department.

3. Cross-Cutting System Requirements

These requirements apply to the system as a whole rather than to only one stakeholder.

SR1 — Incident Location

Requirement:

The system shall associate every reported incident with a specific geographic location using an address and/or geographic coordinates.

Type: Functional

Related entities:

INCIDENT → INCIDENT_LOCATION

Explanation

Every incident must have a location because city authorities need to know where the problem occurred.

The location can contain:

- Building number
 - Street
 - District
 - City
 - Postal code
 - Latitude
 - Longitude
-

SR2 — Service Area

Requirement:

The system shall be able to link an incident to a specific service area for routing and reporting.

Type: Functional

Related entities:

INCIDENT ↔ SERVICE_AREA

resolved through:

INCIDENT_AREA

Explanation

A service area represents a geographic or administrative area served by the city.

The junction table allows incidents to be associated with service areas without duplicating service-area information inside the incident table.

SR3 — Maintain Incident Responsibility History

Requirement:

The system shall preserve a complete record of who created, validated, and assigned each incident.

Type: Data/Integrity

Related attributes:

- INCIDENT.CitizenID
- VALIDATION.OperatorID
- INCIDENT_ASSIGNMENT.OperatorID

Explanation

The system maintains accountability by recording the person responsible for each important step.

SR4 — Every Incident Has a Status

Requirement:

The system shall ensure that every incident always has a status, including immediately after creation. The initial status is **Reported**.

Type: Data Integrity

Related entity:

INCIDENT → INCIDENT_STATUS

Explanation

An incident cannot exist without a status.

The status allows the system to understand the current stage of the incident lifecycle.

SR5 — Team Department Relationship

Requirement:

The system shall allow a response team to belong to exactly one department.

Type: Data Integrity

Relationship:

DEPARTMENT 1 : N RESPONSE_TEAM

Explanation

One department can have multiple response teams, but each response team belongs to one department.

4. Non-Functional Requirements

Non-functional requirements describe **how the system should operate**, rather than specific functions it provides.

NFR1 — Accessibility

The system shall be accessible to citizens through a mobile-friendly interface.

NFR2 — Notification Speed

The system shall notify citizens about incident status changes within a reasonable period of time, preferably near real-time.

NFR3 — Role-Based Security

The system shall restrict access to administrative and operational functions according to the user's role.

The main roles include:

- Citizen
- Operator
- Response Team Member
- Department Manager
- City Administrator

The role system is implemented through:

USER_ACCOUNT + ROLE + ROLE_ASSIGNMENT

NFR4 — Historical Data

The system shall preserve historical information about resolved incidents for reporting and analytics.

Relevant entities include:

- INCIDENT_STATUS_HISTORY
- PERFORMANCE_RECORD

NFR5 — Referential Integrity

The database shall enforce relationships between related entities using foreign-key constraints.

This ensures that records cannot reference non-existing records in related tables.

5. Requirements Traceability

Requirements traceability means that every requirement should be connected to the database design.

The project uses the following principle:

Requirement → Entity/Relationship → Database Table

For example:

Example 1

Requirement: A citizen reports an incident.

↓

CITIZEN → INCIDENT

↓

INCIDENT table contains CitizenID.

Example 2

Requirement: An incident can contain multiple photos.

↓

INCIDENT → INCIDENT_EVIDENCE

↓

INCIDENT_EVIDENCE stores the evidence belonging to each incident.

Example 3

Requirement: An operator assigns an incident to a response team.

↓

OPERATOR → INCIDENT_ASSIGNMENT ← RESPONSE_TEAM

↓

INCIDENT_ASSIGNMENT stores the assignment details.

This traceability allows the project team to demonstrate that the database design directly supports the business requirements.

PART 2 — Relational Schema & Data Dictionary (R#3)

1. Introduction to the Relational Schema

The relational schema describes how the system's information is stored in database tables.

Each table represents an entity or relationship from the ERD.

The database uses:

- **Primary Keys (PK)** to uniquely identify records.
- **Foreign Keys (FK)** to connect related tables.
- **Not Null (NN)** constraints for required values.
- **Unique (U)** constraints to prevent duplicate values.
- **Check constraints** to restrict values to valid ranges or combinations.

Important terms

PK — Primary Key

Uniquely identifies each row.

Example:

`UserID` uniquely identifies a user.

FK — Foreign Key

Connects one table to another table.

Example:

`CitizenID` in `INCIDENT` refers to a citizen in `CITIZEN`.

NN — Not Null

Means the value is required.

U — Unique

Means duplicate values are not allowed.

2. Security & Access

USER_ACCOUNT

This table stores login and account information for system users.

Column	Type	Notes
UserID	INT	PK
Username	VARCHAR(50)	NN, U
Password	VARCHAR(255)	NN , stored as a hash
Email	VARCHAR(100)	NN, U
AccountStatus	VARCHAR(20)	Example: Active, Suspended
CreatedDate	DATETIME	NN
LastLoginDate	DATETIME	Nullable

Purpose

`USER_ACCOUNT` provides the common authentication information used by different types of users.

ROLE

Stores the roles that can be assigned to users.

Column	Type	Notes
RoleID	INT	PK
RoleName	VARCHAR(50)	NN, U
Description	VARCHAR(255)	Description of the role

Examples of roles include:

- Citizen
- Operator
- Response Team Member

- Department Manager
- City Administrator

ROLE_ASSIGNMENT

This table resolves the **M:N relationship** between `USER_ACCOUNT` and `ROLE`.

A user can have multiple roles, and a role can be assigned to multiple users.

Column	Type	Notes
RoleAssignmentID	INT	PK
UserID	INT	FK → USER_ACCOUNT, NN
RoleID	INT	FK → ROLE, NN
AssignedDate	DATE	NN
EndDate	DATE	Nullable
AssignmentStatus	VARCHAR(20)	Current status

3. Citizens

CITIZEN

Stores information about citizens registered in the system.

Column	Type	Notes
CitizenID	INT	PK
UserID	INT	FK → USER_ACCOUNT, nullable, U
FirstName	VARCHAR(50)	NN
MiddleName	VARCHAR(50)	Nullable
LastName	VARCHAR(50)	NN
DateOfBirth	DATE	Nullable

Column	Type	Notes
Nationality	VARCHAR(50)	Nullable
RegistrationDate	DATE	NN

The citizen's login information is stored in `USER_ACCOUNT`.

CITIZEN_PHONE

Stores multiple phone numbers for a citizen.

This table resolves the **multivalued PhoneNumber attribute**.

Column	Type	Notes
CitizenID	INT	Composite PK, FK → CITIZEN
PhoneNumber	VARCHAR(20)	Composite PK

A citizen can therefore have more than one phone number.

CITIZEN_ADDRESS

Stores citizen addresses.

Column	Type	Notes
AddressID	INT	PK
CitizenID	INT	FK → CITIZEN, NN
BuildingNumber	VARCHAR(10)	
Street	VARCHAR(100)	
District	VARCHAR(50)	
City	VARCHAR(50)	
PostalCode	VARCHAR(15)	

Column	Type	Notes
AddressType	VARCHAR(20)	Example: Home, Work

4. Incident Core

INCIDENT

This is the **central table of the system**.

It stores the main information about every reported incident.

Column	Type	Notes
IncidentID	INT	PK
CitizenID	INT	FK → CITIZEN, NN
CategoryID	INT	FK → INCIDENT_CATEGORY, NN
TypeID	INT	FK → INCIDENT_TYPE, NN
LocationID	INT	FK → INCIDENT_LOCATION, NN
StatusID	INT	FK → INCIDENT_STATUS, NN
SeverityID	INT	FK → SEVERITY, NN
PriorityID	INT	FK → PRIORITY, NN
Title	VARCHAR(150)	NN
Description	TEXT	Incident details
ReportedDateTime	DATETIME	NN
ResolvedDateTime	DATETIME	Nullable

Explanation

The incident table connects the report to:

- The citizen who reported it
- Its category

- Its type
- Its location
- Its current status
- Its severity
- Its priority

Derived attributes

`ResponseTime` and `ResolutionTime` are **derived attributes**.

They are calculated when needed rather than stored as separate columns.

For example:

ResolutionTime = ResolvedDateTime – ReportedDateTime

INCIDENT_CATEGORY

Defines the broad category of an incident.

Column	Type	Notes
CategoryID	INT	PK
CategoryName	VARCHAR(50)	NN
Description	VARCHAR(255)	Description

INCIDENT_TYPE

Defines the specific type of incident.

Column	Type	Notes
TypeID	INT	PK
TypeName	VARCHAR(50)	NN
Description	VARCHAR(255)	Description

Category vs Type

A **category** is a broader classification.

A **type** describes the specific incident.

For example:

Category: Transportation

Type: Traffic Accident

INCIDENT_LOCATION

Stores the geographic location where an incident occurred.

Column	Type	Notes
LocationID	INT	PK
BuildingNumber	VARCHAR(10)	
Street	VARCHAR(100)	
District	VARCHAR(50)	
City	VARCHAR(50)	
PostalCode	VARCHAR(15)	
Latitude	DECIMAL(9,6)	Geographic coordinate
Longitude	DECIMAL(9,6)	Geographic coordinate

SERVICE_AREA

Represents a geographic service area.

Column	Type	Notes
AreaID	INT	PK
AreaName	VARCHAR(50)	NN
District	VARCHAR(50)	
City	VARCHAR(50)	

Column	Type	Notes
PostalCode	VARCHAR(15)	
Description	VARCHAR(255)	

INCIDENT_AREA

Resolves the relationship between `INCIDENT` and `SERVICE_AREA`.

Column	Type	Notes
IncidentAreaID	INT	PK
IncidentID	INT	FK → INCIDENT , NN
AreaID	INT	FK → SERVICE_AREA , NN
AssignedDate	DATE	Date of association

INCIDENT_STATUS

Stores the possible statuses of an incident.

Column	Type	Notes
StatusID	INT	PK
StatusName	VARCHAR(30)	NN
Description	VARCHAR(255)	Description

Typical statuses include:

1. Reported
2. Validated
3. Assigned
4. In Progress
5. Resolved
6. Closed

SEVERITY

Defines how serious an incident is.

Column	Type	Notes
SeverityID	INT	PK
SeverityName	VARCHAR(20)	NN
Description	VARCHAR(255)	Description

Typical values:

- Low
- Medium
- High
- Critical

PRIORITY

Defines how urgently an incident should be handled.

Column	Type	Notes
PriorityID	INT	PK
PriorityName	VARCHAR(20)	NN
PriorityLevel	INT	NN
Description	VARCHAR(255)	Description

5. Weak Entities

A **weak entity** is an entity that depends on another entity for its identification or existence.

The project contains three important weak/associative structures.

INCIDENT_EVIDENCE

Stores files associated with an incident.

Column	Type	Notes
IncidentID	INT	Composite PK, FK → INCIDENT
EvidenceNumber	INT	Composite PK , partial key
FileName	VARCHAR(150)	NN
FileType	VARCHAR(20)	
FilePath	VARCHAR(255)	NN
UploadDateTime	DATETIME	NN
EvidenceType	VARCHAR(30)	
Notes	VARCHAR(255)	

Why is it weak?

An evidence record is identified together with its incident.

For example:

```
Incident 100 + Evidence 1
```

```
Incident 100 + Evidence 2
```

INCIDENT_STATUS_HISTORY

Stores the history of status changes for an incident.

Column	Type	Notes
IncidentID	INT	Composite PK, FK → INCIDENT
HistoryNumber	INT	Composite PK , partial key
StatusID	INT	FK → INCIDENT_STATUS, NN
ChangeDateTime	DATETIME	NN
Notes	VARCHAR(255)	

Purpose

This table ensures that the system does not only know the current status. It can also determine how the incident progressed over time.

INCIDENT_COMMENT

Stores comments made by employees regarding incidents.

Column	Type	Notes
CommentID	INT	PK
IncidentID	INT	FK → INCIDENT, NN
EmployeeID	INT	FK → EMPLOYEE, NN
CommentText	TEXT	NN
CommentDateTime	DATETIME	NN
CommentType	VARCHAR(20)	Type of comment

6. Organization

DEPARTMENT

Represents a city department.

Column	Type	Notes
DepartmentID	INT	PK
AdminID	INT	FK → CITY_ADMINISTRATOR, NN
DepartmentName	VARCHAR(50)	NN
Description	VARCHAR(255)	
Phone	VARCHAR(20)	
Email	VARCHAR(100)	

Column	Type	Notes
Location	VARCHAR(100)	

EMPLOYEE

Stores information about employees.

Column	Type	Notes
EmployeeID	INT	PK
UserID	INT	FK → USER_ACCOUNT, nullable, U
DepartmentID	INT	FK → DEPARTMENT, NN
FirstName	VARCHAR(50)	NN
MiddleName	VARCHAR(50)	Nullable
LastName	VARCHAR(50)	NN
Phone	VARCHAR(20)	
HireDate	DATE	

Login information such as email and account status is maintained through `USER_ACCOUNT`.

OPERATOR

Stores information specific to employees who perform operator duties.

Column	Type	Notes
OperatorID	INT	PK
EmployeeID	INT	FK → EMPLOYEE, NN, U
EmployeeNumber	VARCHAR(20)	
Certification	VARCHAR(100)	

Column	Type	Notes
OperatorStatus	VARCHAR(20)	

RESPONSE_TEAM

Stores city response teams.

Column	Type	Notes
TeamID	INT	PK
DepartmentID	INT	FK → DEPARTMENT, NN
TeamLeaderID	INT	FK → EMPLOYEE, nullable
TeamName	VARCHAR(50)	NN
TeamType	VARCHAR(50)	
AvailabilityStatus	VARCHAR(20)	

Important relationship

A department can have many response teams.

DEPARTMENT 1 : N RESPONSE_TEAM

Each response team belongs to exactly one department.

TEAM_MEMBER

Resolves the M:N relationship between employees and response teams.

Column	Type	Notes
TeamID	INT	Composite PK, FK → RESPONSE_TEAM
MemberNumber	INT	Composite PK, partial key
EmployeeID	INT	FK → EMPLOYEE, NN

Column	Type	Notes
JoinDate	DATE	NN
LeaveDate	DATE	Nullable
TeamRole	VARCHAR(30)	

This allows employees to be associated with teams while storing additional information about their membership.

TEAM_SKILL

Stores skills that response teams can have.

Column	Type	Notes
SkillID	INT	PK
SkillName	VARCHAR(50)	NN
SkillDescription	VARCHAR(255)	
SkillLevel	VARCHAR(20)	

TEAM_HAS_SKILL

Resolves the M:N relationship between response teams and skills.

Column	Type	Notes
TeamID	INT	Composite PK, FK → RESPONSE_TEAM
SkillID	INT	Composite PK, FK → TEAM_SKILL

7. Assignment & Response

INCIDENT_ASSIGNMENT

Stores the assignment of an incident to a response team.

Column	Type	Notes
AssignmentID	INT	PK
IncidentID	INT	FK → INCIDENT, NN
TeamID	INT	FK → RESPONSE_TEAM, NN
DepartmentID	INT	FK → DEPARTMENT, NN
OperatorID	INT	FK → OPERATOR, NN
AssignmentDateTime	DATETIME	NN
AcceptedDateTime	DATETIME	Nullable
CompletionDateTime	DATETIME	Nullable
AssignmentStatus	VARCHAR(20)	
AssignmentMethod	VARCHAR(20)	
AssignmentNotes	VARCHAR(255)	

Purpose

This table is important because it records the complete assignment process.

It answers questions such as:

- Which incident was assigned?
- Which team received it?
- Which department does the team belong to?
- Which operator made the assignment?
- When was it assigned?
- When did the team accept it?
- When was the work completed?

VALIDATION

Stores the result of an operator's validation of an incident.

Column	Type	Notes
ValidationID	INT	PK
IncidentID	INT	FK → INCIDENT, NN, U
OperatorID	INT	FK → OPERATOR, NN
ValidationDateTime	DATETIME	NN
ValidationResult	VARCHAR(20)	
ValidationNotes	VARCHAR(255)	

The unique `IncidentID` means that the validation record is associated with a specific incident.

8. Resources

The system also manages resources used by response teams.

Resources include vehicles and equipment.

RESOURCE_TYPE

Defines resource types.

Column	Type	Notes
ResourceTypeID	INT	PK
TypeName	VARCHAR(50)	NN
Description	VARCHAR(255)	

VEHICLE

Stores information about vehicles used by response teams.

Column	Type	Notes
VehicleID	INT	PK
ResourceTypeID	INT	FK → RESOURCE_TYPE, NN
TeamID	INT	FK → RESPONSE_TEAM, nullable
PlateNumber	VARCHAR(20)	U
Model	VARCHAR(50)	
Status	VARCHAR(20)	
PurchaseDate	DATE	

The vehicle type is provided through `RESOURCE_TYPE`.

EQUIPMENT

Stores equipment used by response teams.

Column	Type	Notes
EquipmentID	INT	PK
ResourceTypeID	INT	FK → RESOURCE_TYPE, NN
TeamID	INT	FK → RESPONSE_TEAM, nullable
EquipmentName	VARCHAR(50)	NN
Quantity	INT	
Condition	VARCHAR(20)	
Status	VARCHAR(20)	

The equipment type is provided through `RESOURCE_TYPE`.

RESOURCE_ASSIGNMENT

Connects resources to incident assignments.

Column	Type	Notes
ResourceAssignmentID	INT	PK
AssignmentID	INT	FK → INCIDENT_ASSIGNMENT, NN
VehicleID	INT	FK → VEHICLE, nullable
EquipmentID	INT	FK → EQUIPMENT, nullable
AssignmentDateTime	DATETIME	NN
ReturnDateTime	DATETIME	Nullable
Quantity	INT	
Notes	VARCHAR(255)	

Important rule

A resource assignment uses **exactly one** resource type:

- Either a vehicle
- Or equipment

It should not contain both at the same time.

This rule is enforced using a database `CHECK` constraint.

9. Communication

NOTIFICATION

Stores notifications sent to citizens.

Column	Type	Notes
NotificationID	INT	PK
CitizenID	INT	FK → CITIZEN, NN
IncidentID	INT	FK → INCIDENT, NN
NotificationType	VARCHAR(30)	

Column	Type	Notes
Message	VARCHAR(255)	NN
SentDateTime	DATETIME	NN
ReadDateTime	DATETIME	Nullable
NotificationStatus	VARCHAR(20)	

CITIZEN_FEEDBACK

Stores citizen feedback after incident resolution.

Column	Type	Notes
FeedbackID	INT	PK
CitizenID	INT	FK → CITIZEN, NN
IncidentID	INT	FK → INCIDENT, NN, U
Rating	INT	NN, CHECK 1–5
Comments	VARCHAR(255)	
FeedbackDate	DATE	NN

The rating must be between **1 and 5**.

The unique `IncidentID` ensures that an incident has one feedback record.

10. Performance & Reporting

PERFORMANCE_RECORD

Stores performance measurements related to incident handling.

Column	Type	Notes
PerformanceID	INT	PK
TeamID	INT	FK → RESPONSE_TEAM, nullable

Column	Type	Notes
DepartmentID	INT	FK → DEPARTMENT, nullable
IncidentID	INT	FK → INCIDENT, NN
EvaluationDate	DATE	NN
ResponseTime	DECIMAL(10,2)	
ResolutionTime	DECIMAL(10,2)	
IncidentsHandled	INT	
IncidentsResolved	INT	
AverageRating	DECIMAL(3,2)	

Purpose

This table supports performance analysis at the team, department, and incident levels.

It can be used to determine:

- How quickly teams respond
- How quickly incidents are resolved
- How many incidents were handled
- How many incidents were resolved
- Average citizen ratings

PERFORMANCE_REPORT

Stores generated performance reports.

Column	Type	Notes
ReportID	INT	PK
DepartmentID	INT	FK → DEPARTMENT, NN
EmployeeID	INT	FK → EMPLOYEE, NN
ReportType	VARCHAR(30)	

Column	Type	Notes
ReportTitle	VARCHAR(150)	NN
GeneratedDateTime	DATETIME	NN
ReportPeriodStart	DATE	
ReportPeriodEnd	DATE	
ReportDescription	TEXT	

A report is associated with the department it describes and the employee who prepared it.

11. Administration

CITY_ADMINISTRATOR

Stores information specific to city administrators.

Column	Type	Notes
AdminID	INT	PK
UserID	INT	FK → USER_ACCOUNT, nullable, U
FirstName	VARCHAR(50)	NN
LastName	VARCHAR(50)	NN
Phone	VARCHAR(20)	

Login information is maintained through `USER_ACCOUNT`.

CATEGORY_DEPARTMENT

Resolves the M:N relationship between incident categories and departments.

Column	Type	Notes
CategoryDepartmentID	INT	PK
CategoryID	INT	FK → INCIDENT_CATEGORY, NN

Column	Type	Notes
DepartmentID	INT	FK → DEPARTMENT, NN
AssignmentDate	DATE	
IsPrimaryDepartment	BOOLEAN/BIT	Indicates the primary department

Example

A category such as **Road Damage** could be associated with a Roads Department.

Another category could potentially be handled by more than one department.

The `IsPrimaryDepartment` attribute can identify which department is normally responsible.

12. Database Design Concepts Used

The database applies several important relational database concepts.

12.1 Primary Keys

Every major entity has a primary key.

Examples:

- `UserID`
- `CitizenID`
- `IncidentID`
- `DepartmentID`
- `EmployeeID`
- `TeamID`

The primary key uniquely identifies each record.

12.2 Foreign Keys

Foreign keys connect related entities.

For example:

`INCIDENT.CitizenID`

references:

`CITIZEN.CitizenID`

This means that an incident is connected to the citizen who reported it.

12.3 One-to-Many Relationships

Example:

DEPARTMENT → RESPONSE_TEAM

One department can contain many response teams, while each response team belongs to one department.

12.4 Many-to-Many Relationships

M:N relationships are implemented using junction tables.

Examples:

USER_ACCOUNT ↔ ROLE

Resolved by:

`ROLE_ASSIGNMENT`

EMPLOYEE ↔ RESPONSE_TEAM

Resolved by:

`TEAM_MEMBER`

RESPONSE_TEAM ↔ TEAM_SKILL

Resolved by:

`TEAM_HAS_SKILL`

INCIDENT_CATEGORY ↔ DEPARTMENT

Resolved by:

`CATEGORY_DEPARTMENT`

12.5 Multivalued Attributes

A citizen may have multiple phone numbers.

Instead of storing several phone numbers in one column, the design uses:

CITIZEN_PHONE

This keeps the database normalized.

12.6 Composite Attributes

The Name attribute is divided into:

- FirstName
- MiddleName
- LastName

This structure is used for:

- CITIZEN
 - EMPLOYEE
-

12.7 Derived Attributes

Some values can be calculated from existing data.

For example:

ResponseTime

and

ResolutionTime

can be calculated using stored dates and times.

They are therefore treated as derived values rather than duplicate stored information.

13. Complete Database Structure Summary

The final database contains **36 tables**.

These represent the original entities and the required junction structure.

Security & Access

1. USER_ACCOUNT
2. ROLE
3. ROLE_ASSIGNMENT

Citizens

4. CITIZEN
5. CITIZEN_PHONE
6. CITIZEN_ADDRESS

Incident Core

7. INCIDENT
8. INCIDENT_CATEGORY
9. INCIDENT_TYPE
10. INCIDENT_LOCATION
11. SERVICE_AREA
12. INCIDENT_AREA
13. INCIDENT_STATUS
14. SEVERITY
15. PRIORITY

Evidence & History

16. INCIDENT_EVIDENCE
17. INCIDENT_STATUS_HISTORY
18. INCIDENT_COMMENT

Organization

19. DEPARTMENT
20. EMPLOYEE
21. OPERATOR
22. RESPONSE_TEAM
23. TEAM_MEMBER
24. TEAM_SKILL
25. TEAM_HAS_SKILL

Assignment & Response

26. INCIDENT_ASSIGNMENT
27. VALIDATION

Resources

- 28. RESOURCE_TYPE
- 29. VEHICLE
- 30. EQUIPMENT
- 31. RESOURCE_ASSIGNMENT

Communication

- 32. NOTIFICATION
- 33. CITIZEN_FEEDBACK

Performance & Reporting

- 34. PERFORMANCE_RECORD
- 35. PERFORMANCE_REPORT

Administration

- 36. CITY_ADMINISTRATOR
- 37. CATEGORY_DEPARTMENT

Note: The table count and grouping should be understood according to the project's ERD/schema version; the design includes the full set of entities and junction tables described above.

14. Important Design Rules

The database follows several important rules to maintain consistency.

Rule 1 — Every incident has a reporter

Every incident is connected to a citizen through:

INCIDENT.CitizenID

Rule 2 — Every incident has a location

An incident is associated with:

INCIDENT_LOCATION

Rule 3 — Every incident has a category and type

This allows the system to classify incidents consistently.

Rule 4 — Every incident has a status

The incident lifecycle begins with:

Reported

and can progress through statuses such as:

Validated → Assigned → In Progress → Resolved → Closed

Rule 5 — Incidents can have multiple evidence files

The relationship is:

INCIDENT 1 : N INCIDENT_EVIDENCE

Rule 6 — Incidents can have status history

Every status change can be recorded in:

INCIDENT_STATUS_HISTORY

Rule 7 — Teams belong to departments

The relationship is:

DEPARTMENT 1 : N RESPONSE_TEAM

Rule 8 — Teams can have multiple members

Employees and teams are connected through:

TEAM_MEMBER

Rule 9 — Teams can have multiple skills

Teams and skills are connected through:

TEAM_HAS_SKILL

Rule 10 — Resources belong to assignments

Vehicles and equipment can be connected to an incident assignment through:

`RESOURCE_ASSIGNMENT`

Rule 11 — Only one resource type is assigned per resource-assignment record

A `RESOURCE_ASSIGNMENT` record contains either:

`VehicleID`

or

`EquipmentID`

Rule 12 — Citizen feedback uses a 1–5 rating

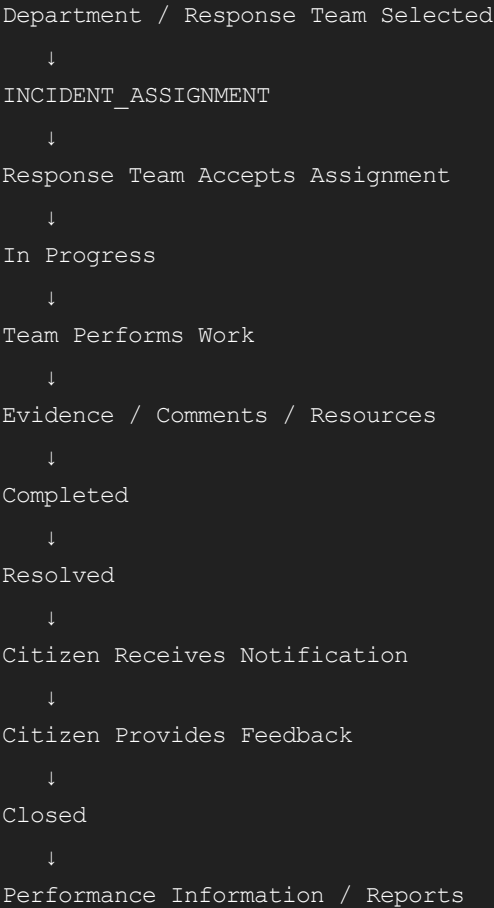
The `Rating` value is restricted to the range:

1–5

15. Overall Incident Lifecycle

The complete incident process can be understood as follows:

```
Citizen
  ↓
Reports Incident
  ↓
INCIDENT
  ↓
Status = Reported
  ↓
Operator Reviews Incident
  ↓
Validation
  ↓
Validated
  ↓
Severity + Priority
  ↓
```



This lifecycle shows how the different database tables work together rather than operating as isolated tables.

16. Final Traceability Summary

The database design is directly connected to the requirements defined in R#1.

Requirement Area	Main Database Components
Citizen reporting	CITIZEN , INCIDENT
Incident classification	INCIDENT_CATEGORY , INCIDENT_TYPE
Location	INCIDENT_LOCATION , SERVICE_AREA , INCIDENT_AREA
Status tracking	INCIDENT_STATUS , INCIDENT_STATUS_HISTORY
Evidence	INCIDENT_EVIDENCE
Validation	VALIDATION , OPERATOR
Severity & priority	SEVERITY , PRIORITY
Department selection	CATEGORY_DEPARTMENT , DEPARTMENT

Requirement Area	Main Database Components
Response teams	RESPONSE_TEAM , TEAM_MEMBER , TEAM_SKILL , TEAM_HAS_SKILL
Assignment	INCIDENT_ASSIGNMENT
Resources	RESOURCE_TYPE , VEHICLE , EQUIPMENT , RESOURCE_ASSIGNMENT
Notifications	NOTIFICATION
Citizen feedback	CITIZEN_FEEDBACK
Employee management	EMPLOYEE , DEPARTMENT
Authentication	USER_ACCOUNT
Authorization	ROLE , ROLE_ASSIGNMENT
Performance	PERFORMANCE_RECORD
Reports	PERFORMANCE_REPORT
Administration	CITY_ADMINISTRATOR

17. Final Project Design

The Smart City Incident Management System combines **citizen services, incident management, organizational management, resource management, communication, and performance reporting** into one relational database.

The overall design can be summarized as:

Citizen reports an incident

- Incident is classified and located
- Operator validates it
- Severity and priority are determined
- Appropriate department/team is selected

- **Incident is assigned**
- **Response team handles the incident**
- **Resources and evidence are recorded**
- **Incident status is updated**
- **Citizen receives notifications**
- **Incident is resolved and closed**
- **Citizen provides feedback**
- **Performance information is recorded**
- **Managers and administrators generate reports**

This structure ensures that the system can track an incident from its initial report until final resolution while maintaining the people, teams, departments, resources, evidence, history, and performance information connected to that incident.

Document Status: FINAL — Requirements and Relational Schema Reference